

Reviewer Pack — Minimal Order of Formal Grammar Complexity and Circuit Monot...

Entry 6ef447db8812 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 16:56:19 UTC

Minimal Order of Formal Grammar Complexity and Circuit Monotone Width Inequality

Entry ID: 6ef447db8812 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 16:56:19 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory (Grammar Complexity) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every CNF formula φ with m clauses and n variables, the minimal number of distinct formal grammars required to generate all possible satisfying assignments of φ is linearly correlated with its circuit monotone width $w(\varphi)$, such that $\text{MinimalGrammarComplexity}(\varphi) = \Theta(w(\varphi))$

Rationale (proposer's reasoning):

Formal grammar complexity captures the combinatorial structure of a language, while circuit monotone width measures the simplicity of a function in terms of its truth table. This conjecture suggests that understanding the complexity of generating all satisfying assignments is related to the structural simplicity of the corresponding boolean function.

Taxonomy category: FormalLanguageGrammarComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 60f3f99ce0c827dd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF formula φ , if $\text{MinimalGrammarComplexity}(\varphi) / w(\varphi)$ is within $[0.5, 2]$ with Pearson's $r \geq 0.7$ for at least 25/30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(m, n):
        clauses = []
        for _ in range(m):
            clause = set(random.sample(range(1, n+1), 2))
            if random.choice([True, False]):
                clause = {x for x in clause}
            else:
                clause = {-x for x in clause}
            clauses.append(clause)
        return clauses

    def circuit_monotone_width(cnf):
        width = 0
        for clause in cnf:
            if all(abs(x) in assignment for x in clause):
                width += 1
        return width

    def minimal_grammar_complexity(cnf):
        grammar = {}
        for clause in cnf:
            for literal in clause:
                if literal not in grammar:
                    grammar[literal] = set()
                for other_literal in clause:
                    if other_literal != literal:
                        grammar[literal].add(other_literal)
        return len(grammar)

    n_max = 0
    instances_tested = 0
```

```

total_metric_value = 0

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):
        m = random.randint(1, min(n * (n - 1) // 2, 10))
        cnf = generate_cnf(m, n)
        width = circuit_monotone_width(cnf)
        complexity = minimal_grammar_complexity(cnf)
        instances_tested += 1
        total_metric_value += complexity / width
        n_max = max(n_max, n)

metric_value = total_metric_value / instances_tested
conjecture_holds = abs(metric_value - 1) < 0.5 and instances_tested >= 30
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "MinimalGrammarComplexity/WidthRatio",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.2f} std={std_metric_value:.2f} support_
    elif sum(1 for result in results if not result["conjecture_holds"]) / len(results) >= 0.8:
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76293285.py", line 82, in <module>

```

trial_result = run_trial(seed)
^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76293285.py", line 58, in run_trial
    width = circuit_monotone_width(cnf)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76293285.py", line 35, in circuit_monot
    if all(abs(x) in assignment for x in clause):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76293285.py", line 35, in <genexpr>
    if all(abs(x) in assignment for x in clause):
    ^^^^^^^^^
NameError: name 'assignment' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Review and debug the test code to ensure it can run to completion and produce the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14665
2	preregistration	ollama_remote	glm4:latest	0	0	9798
3	novelty	ollama_remote	glm4:latest	0	0	8558
4	novelty	ollama_remote	glm4:latest	0	0	9074
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47739
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10276
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9358
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9577
9	judge	ollama_remote	glm4:latest	0	0	19524

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138569 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/6ef447db8812.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/6ef447db8812.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6ef447db8812.tar.gz` (if generated)